

Python 语言程序设计课程内容

适用对象：高等院校非计算机专业或 Python 入门课程 | 建议学时：48 学时

一、课程性质与总体定位

《Python 语言程序设计》是一门面向程序设计基础、数据处理与问题求解能力培养的入门课程。课程以 Python 语言为载体，强调从“读懂程序、写出程序、调试程序、用程序解决实际问题”四个层面逐步推进，使学生掌握计算思维的基本方法，并能将程序设计能力迁移到学习、科研和业务场景中。

本课程不以语法记忆为唯一目标，而是把语法、算法、数据结构雏形和项目实践组织在连续任务中。教学过程应重视示例讲解、课堂练习、错误分析和小型项目迭代，让学生在真实编码过程中理解变量、控制结构、函数、文件、异常和常用库的意义。

二、教学目标

- 知识目标：理解 Python 程序的基本结构，掌握变量、数据类型、表达式、分支、循环、函数、组合数据类型、文件读写和异常处理等核心知识。
- 能力目标：能够根据问题描述进行需求分解，设计基本算法流程，编写可运行、可调试、可复用的 Python 程序。
- 素养目标：形成规范编码、主动测试、阅读文档和持续改进的习惯，理解程序设计在数据分析、自动化办公和智能应用中的基础作用。

三、课程内容模块

课程内容建议分为六个模块，采用“概念讲授 + 示例演示 + 课堂任务 + 综合练习”的组织方式。各模块之间既保持知识递进，也保留可实践的应用场景。

模块	核心内容	实践任务	建议学时
Python 入门与环境	语言特点、开发环境、程序执行流程、输入输出、注释与编码规范	完成第一个交互式程序	6
基本语法与控制结构	变量、数据类型、运算符、分支语句、循环语句、循环控制	设计成绩判断、猜数字等程序	10
函数与模块化设计	函数定义、参数传递、返回值、作用域、模块导入与简单复用	将重复逻辑封装为函数库	8
组合数据类型	字符串、列表、元组、字典、集合及其常用	实现文本统计和简易通讯录	8

	操作		
文件、异常与常用库	文件读写、异常处理、random、math、datetime、第三方库初识	批量处理 CSV 或文本数据	8
综合项目实践	需求分析、流程设计、代码实现、测试调试、结果展示	完成一个小型数据处理或自动化项目	8

四、教学重点与难点

教学重点包括控制结构的选择、函数化思维、组合数据类型的灵活使用、文件处理流程以及程序调试方法。教学中应避免学生只会照抄示例而不能独立分析问题，可通过变式练习和错误代码诊断提升理解深度。

教学难点主要集中在循环嵌套、函数参数、列表与字典的综合应用、异常处理逻辑以及项目任务拆分。教师可采用流程图、代码追踪表和分阶段提交的方式，把抽象逻辑转化为可观察的步骤。

五、教学实施建议

1. 课前准备：发布简短导学材料和环境检查任务，确保学生能够独立运行 Python 程序。
2. 课堂组织：每次课围绕一个可完成的小问题展开，先给出需求，再讲解关键语法，最后安排限时编码。
3. 练习反馈：对典型错误进行集中讲评，如变量命名混乱、循环边界错误、缩进错误、文件路径错误等。
4. 项目推进：综合项目采用分阶段验收，包括需求说明、核心函数、数据读写、异常处理和最终展示。

六、考核方式与学习评价

课程评价建议采用过程性评价与终结性评价相结合的方式。平时成绩可关注课堂练习、作业提交、代码规范和学习参与度；期末评价可采用上机测试或课程项目答辩，重点考查学生分析问题、组织代码和解释程序运行结果的能力。

建议比例为：课堂练习与出勤 20%，阶段作业 30%，综合项目 30%，期末上机测试 20%。评价标准应兼顾程序正确性、代码可读性、测试充分性和问题表达能力。

七、预期学习成果

- 能够独立搭建 Python 开发环境，阅读并运行基础程序。
- 能够使用分支、循环和函数解决常见计算与文本处理问题。
- 能够使用列表、字典等结构组织数据，并完成简单的数据统计任务。
- 能够完成一个结构清晰、具有实际用途的小型 Python 项目。